

Module 11 Lab Session 3: Policy Authorization, Angular Guards & Security Hardening

Session 3 of 3

Exercises Exercises 5, 6, and 7

Integrity Lab tier **Tier 3** — Policy authorization, custom AuthorizationHandler, Angular role guards, rate limiting, security headers, final audit

Story thread

Role-based authorization (`[Authorize(Roles = "Instructor")]`) is insufficient when an instructor attempts to modify a course taught by someone else. You will build a **Resource-Based Authorization Handler** ensuring instructors can only modify courses where they are assigned as lead instructor. Then, you will implement Angular `roleGuard` and structural template checks for clean client UX, and finish by hardening the API against DDoS and browser attacks using rate limiting and security headers middleware.

Exercise 5: Resource-Based Authorization (Course Ownership)

Step 1: Define Custom Authorization Requirement

Create `Authorization/CourseInstructorRequirement.cs`:

```
using Microsoft.AspNetCore.Authorization;

namespace Tms.Api.Authorization;

public class CourseInstructorRequirement : IAuthorizationRequirement { }
```

Step 2: Implement Authorization Handler

Create `Authorization/CourseInstructorHandler.cs`:

```
namespace Tms.Api.Authorization;

public class CourseInstructorHandler :
    AuthorizationHandler<CourseInstructorRequirement, Course>
{ }
```

```

    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        CourseInstructorRequirement requirement,
        Course resource)
    {
        var userId =
context.User.FindFirstValue(ClaimTypes.NameIdentifier);
        var isInstructor = context.User.IsInRole("Instructor");
        var isAdmin = context.User.IsInRole("Admin");

        // Admins can manage any course
        if (isAdmin)
        {
            context.Succeed(requirement);
            return Task.CompletedTask;
        }
        // Instructors can only manage courses where InstructorId
matches their User ID
        if (isInstructor && resource.InstructorId == userId)
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}

```

Step 3: Register Policy in Program.cs

In Program.cs:

```

builder.Services.AddAuthorizationBuilder()
    .AddPolicy("CanEditCourse", policy =>
        policy.Requirements.Add(new CourseInstructorRequirement()));

builder.Services.AddSingleton<IAuthorizationHandler, CourseInstructorHa
ndler>();

```

Step 4: Enforce Policy & Roles in Controller

In Controllers/CourseController.cs, inject IAuthorizationService and enforce the policy:

```

namespace Tms.Api.Controllers;

[Authorize(Roles = "Instructor,Admin")]

```

```

[ApiController]
[Route("api/[controller]")]
public class CourseController : ControllerBase
{
    private readonly TmsDbContext _context;
    private readonly IAuthorizationService _authorizationService;

    public CourseController(TmsDbContext context, IAuthorizationService
authorizationService)
    {
        _context = context;
        _authorizationService = authorizationService;
    }

    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateCourse(int id, [FromBody]
UpdateCourseDto dto)
    {
        var course = await _context.Courses.FindAsync(id);
        if (course == null) return NotFound();

        var authResult = await
_authorizedService.AuthorizeAsync(User, course, "CanEditCourse");
        if (!authResult.Succeeded)
        {
            return Forbid(); // 403 Forbidden when caller doesn't own
the resource
        }

        course.Title = dto.Title;
        await _context.SaveChangesAsync();
        return NoContent(); }
}

```

Exercise 6: Angular Route Protection & Interceptors

Step 1: Update AuthService for Bearer Tokens

Update src/app/services/auth.service.ts (first built in M10 Session 2) to store the JWT access token in memory, decode user profile claims, and expose `getAccessToken()`:

```

export interface TmsUser {
    email: string;
    displayName: string;
    role: string;
}

```

```

}

export interface LoginRequest {
  email: string;
  password: string;
}

export interface AuthResponse {
  accessToken: string;
  refreshToken: string;
}

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private accessToken = signal<string | null>(null);
  currentUser = signal<TmsUser | null>(null);

  getAccessToken(): string | null {
    return this.accessToken();
  }

  hasRole(role: string): boolean {
    const user = this.currentUser();
    return user?.role === role || user?.role === 'Admin';
  }

  async login(credentials: LoginRequest): Promise<void> {
    const res = await firstValueFrom(
      this.http.post<AuthResponse>('/api/auth/login', credentials)
    );

    this.accessToken.set(res.accessToken);

    // Decode user payload from JWT (or fetch /api/auth/me)
    const payload = JSON.parse(atob(res.accessToken.split('.')[1]));
    this.currentUser.set({
      email: payload.email || payload.sub,
      displayName: payload.name || payload.email || 'User',
      role:
payload['http://schemas.microsoft.com/ws/2008/06/identity/claims/role']
|| payload.role || 'Student'
    });
  }

  logout(): void {
    this.accessToken.set(null);
    this.currentUser.set(null);
  }
}

```

Step 2: Functional Role Guard

Create src/app/guards/role.guard.ts:

```
export const roleGuard = (requiredRole: string): CanActivateFn => {
  return () => {
    const auth = inject(AuthService);
    const router = inject(Router);

    if (auth.hasRole(requiredRole)) {
      return true;
    }

    return router.createUrlTree(["/unauthorized"]);
  };
};
```

Apply in app.routes.ts:

```
{
  path: 'admin/courses',
  component: AdminCourseListComponent,
  canActivate: [roleGuard('Admin')]
}
```

Step 3: Conditional UI in Templates

Hide administrative buttons from non-admin users:

```
@if (auth.hasRole('Admin')) {
  <button (click)="deleteCourse(course.id)" class="btn-danger">
    Delete Course
  </button>
}
```

Step 4: HTTP Auth Interceptor

Create src/app/interceptors/jwt.interceptor.ts:

```
export const jwtInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.getAccessToken();

  if (token) {
    const cloned = req.clone({
```

```

        setHeaders: { Authorization: `Bearer ${token}` }
    });
    return next(cloned);
}

return next(req);
};

```

Exercise 7: Security Hardening (Rate Limiting & Headers)

Step 1: Rate Limiting in .NET API

In Program.cs:

```

builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("AuthLimiter", opt =>
    {
        opt.PermitLimit = 5;
        opt.Window = TimeSpan.FromMinutes(1);
        opt.QueueLimit = 0;
    });
});

```

// After UseRouting: if you don't have it already
app.UseRateLimiter();

Apply to AuthController.cs:

```

[EnableRateLimiting("AuthLimiter")]
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginRequest request)

```

Step 2: Security Response Headers Middleware

Add security headers to responses in Program.cs:

```

app.Use(async (context, next) =>
{
    context.Response.Headers.Append("X-Content-Type-Options",
    "nosniff");
    context.Response.Headers.Append("X-Frame-Options", "DENY");
    context.Response.Headers.Append("Referrer-Policy", "strict-origin-
when-cross-origin");
    context.Response.Headers.Append(

```

```
        "Content-Security-Policy",
        "default-src 'self'; script-src 'self'; style-src 'self'
'unsafe-inline';");
    await next();
});
```

Final Security Sprint Verification

Verify your hardened TMS pipeline:

- ☐ **Auth & Lockout:** 5 failed logins triggers 423 Locked.
- ☐ **JWT Bearer:** API requires Authorization: Bearer <token> header on protected routes.
- ☐ **Resource Policy:** Instructor A receives 403 Forbidden attempting to edit Instructor B's course.
- ☐ **Angular Guard & UI:** Unauthorized user attempting /admin/courses routes to /unauthorized.
- ☐ **Rate Limiting:** Exceeding 5 login attempts/min triggers 429 Too Many Requests.
- ☐ **Security Headers:** Response headers include X-Frame-Options: DENY and X-Content-Type-Options: nosniff.